

SmartLogix

Intelligent Decision Support System for Logistics & Supply Chain

System Design & Implementation Document

Technology Stack: Next.js + Supabase

Module: PDSA (Programming, Data Structures and Algorithms)

Course: BSc (Hons) Computing — 26.1

Coursework 1 — Group Project Documentation

Table of Contents

1. Introduction.....	3
2. System Overview	3
2.1 Business Scenario	3
2.2 High-Level Objectives	3
3. Technology Stack	3
4. System Architecture	4
4.1 Architectural Style	4
4.2 Logical Layers.....	4
4.3 System Architecture Diagram (Description).....	5
4.4 Module Interaction.....	5
5. System Users and Their Tasks	5
5.1 Role–Module Mapping	5
6. System Modules Overview	6
7. Detailed Module Design	6
7.1 Intelligent Route Optimization	6
7.2 Intelligent Resource Allocation.....	7
7.3 Network Analysis	7
7.4 Intelligent Decision Module	8
7.5 Optimization Module (Intractable Problem)	8
8. Database Design (Supabase / PostgreSQL)	9
8.1 Entity Relationship Notes	10
9. System Features Summary	10
9.1 Core Platform Features	10
9.2 Module-Specific Features.....	10
Task 1 — Intelligent Route Optimization.....	10
Task 2 — Intelligent Resource Allocation	11
Task 3 — Network Analysis	11
Task 4 — Intelligent Decision Module.....	11
Task 5 — Optimization Module (Multi-stop VRP/TSP)	11
10. End-to-End System Workflow	11
11. Non-Functional Requirements	12
12. Experimental Performance Evaluation Plan.....	12
13. Deployment Architecture	13
14. Conclusion	13

1. Introduction

SmartLogix is a web-based Intelligent Decision Support System (IDSS) developed for a logistics and supply-chain consultancy scenario. The system integrates five independent computational modules — Route Optimization, Resource Allocation, Network Analysis, an Intelligent Decision module, and an Optimization module for an intractable routing problem — into a single, unified application.

This document describes the complete system design, including the chosen technology stack (Next.js and Supabase), the system's users and their tasks, the features delivered by each module, the underlying algorithms and data structures, the database design, and the plan for experimental performance evaluation required by the coursework.

2. System Overview

The system is built as a single Next.js web application with role-based dashboards, backed by a Supabase PostgreSQL database. All five coursework modules operate on a shared set of logistics data — warehouses, vehicles, drivers, orders and suppliers — so that the outputs of one module (e.g., a computed route) feed naturally into another (e.g., resource allocation for that route).

2.1 Business Scenario

A logistics company operates a network of warehouses/distribution hubs connected by roads. Customers place delivery orders that must be routed efficiently, loaded onto capacity-constrained vehicles, and delivered along optimized multi-stop tours. The company also needs to plan cost-efficient infrastructure between hubs and to evaluate backup suppliers when inventory is at risk.

2.2 High-Level Objectives

- Provide dispatchers with tools to compute optimal point-to-point delivery routes.
- Maximize the value of cargo loaded onto weight-constrained vehicles.
- Support strategic decisions on which infrastructure links to build between hubs.
- Automate supplier risk classification to speed up procurement decisions.
- Efficiently plan complex multi-stop delivery tours despite their computational intractability.
- Provide experimental evidence (execution time, memory, scalability) comparing candidate algorithms for each module.

3. Technology Stack

The system is implemented as a modern full-stack web application using Next.js for the frontend and application layer, and Supabase as the backend-as-a-service platform providing the database, authentication, storage and realtime capabilities.

Layer	Technology	Purpose
Frontend	Next.js 14+ (App Router), React, TypeScript, Tailwind CSS, shadcn/ui	Admin dashboard, dispatcher console, driver view, customer portal — all rendered as a single unified web application with role-based routing.
Backend / Compute	Next.js Route Handlers (API routes) and Server Actions running on Node.js	Houses the five DSA modules as server-side services (graph algorithms, DP, tree logic). CPU-heavy jobs (Simulated Annealing, large-graph Dijkstra) can be offloaded to Supabase Edge Functions.
Database	Supabase (managed PostgreSQL)	Stores all relational data: users, warehouses, vehicles, orders, routes, network edges, suppliers, and benchmark logs.
Authentication & Authorization	Supabase Auth + Row Level Security (RLS)	Handles login/session management and enforces per-role data access policies (Admin, Dispatcher, Analyst, Driver, Procurement, Customer).
Realtime Updates	Supabase Realtime (Postgres change subscriptions)	Pushes live order/delivery-status updates to dispatcher and customer dashboards without polling.
File / Report Storage	Supabase Storage	Stores exported PDF/CSV performance reports, route maps snapshots, and supplier documents.
Maps & Visualization	Leaflet.js / Mapbox GL, Chart.js or Recharts	Renders road/hub network maps, optimized routes, and the execution-time / memory benchmark charts required for Chapters 7–9.
Hosting / Deployment	Vercel (Next.js app) + Supabase Cloud (managed backend)	CI/CD via GitHub → Vercel automatic deployments; Supabase manages database backups and scaling.

4. System Architecture

4.1 Architectural Style

SmartLogix follows a modern Jamstack / server-rendered architecture rather than the traditional three-tier Flask/Laravel/MySQL split. Next.js serves both the UI (React Server/Client Components) and the application/API layer (Route Handlers and Server Actions), while Supabase provides a fully managed PostgreSQL database, authentication, row-level security, realtime subscriptions and file storage.

4.2 Logical Layers

- **Presentation Layer** — Next.js App Router pages and React components for each role's dashboard (Admin, Dispatcher, Analyst, Driver, Procurement, Customer).
- **Application / Algorithm Layer** — TypeScript service modules under `/lib/algorithms` implementing Dijkstra's Algorithm, 0/1 Knapsack DP, Prim's Algorithm, the Decision Tree classifier, and Simulated Annealing, exposed via Next.js Route Handlers.
- **Data Layer** — Supabase PostgreSQL tables accessed via the Supabase JS client (and optionally Prisma), protected by Row Level Security policies per role.
- **Cross-Cutting Services** — Supabase Auth (login/session), Supabase Realtime (live order/route status), Supabase Storage (exported reports and supplier documents).

4.3 System Architecture Diagram (Description)

Client Browser → Next.js Application (Vercel) → [Route Handlers / Server Actions calling Algorithm Services] → Supabase (PostgreSQL + Auth + Realtime + Storage). Realtime change events flow back from Supabase to subscribed dashboards (Dispatcher, Driver, Customer) without page reloads. This diagram should be redrawn formally (e.g., in draw.io or Figma) and included as a figure in the Group Report, Chapter 2.

4.4 Module Interaction

The five modules are decoupled services that share the same underlying data model: an order created by a Dispatcher triggers Route Optimization; multiple orders assigned to one vehicle trigger Resource Allocation; the day's full set of stops for a vehicle triggers the Optimization Module; hub/warehouse data is independently analyzed by the Network Analysis module; and supplier data is independently classified by the Decision module. All modules write their timing/memory statistics to a shared benchmark_logs table for the evaluation chapters.

5. System Users and Their Tasks

SmartLogix supports six distinct user roles, each with a tailored dashboard enforced through Supabase Auth roles and Row Level Security policies.

User / Role	Key Tasks
System Administrator	Manage user accounts and role permissions; configure warehouses, vehicles and drivers; monitor overall system health; view system-wide analytics and benchmark reports; manage master data (hubs, network edges).
Logistics / Dispatch Manager	Create and manage customer orders; trigger Route Optimization for point-to-point deliveries; run Resource Allocation to assign cargo to vehicles within weight limits; launch multi-stop Route Optimization (Simulated Annealing) for daily delivery runs; monitor live order and delivery status.
Supply Chain Analyst	Run Network Analysis (MST) to evaluate warehouse/hub connectivity and infrastructure cost; review supplier risk classification results from the Intelligent Decision module; generate and export comparative performance reports (time/memory/scalability charts).
Driver / Field Operator	View assigned delivery route and stop sequence on a mobile-friendly dashboard; update delivery status per stop (arrived, delivered, failed); report real-time location (optional live tracking).
Procurement Officer	Register and update supplier records (cost, delivery speed, historical defect rate); review Decision Tree classification output when selecting backup suppliers; approve or override system recommendations.
Customer / Client (self-service portal)	Place delivery orders; track order and delivery status; view estimated delivery time generated by the Route Optimization module.

5.1 Role–Module Mapping

- System Administrator — configures master data used by all modules (warehouses, vehicles, network edges).
- Logistics / Dispatch Manager — primary user of Route Optimization, Resource Allocation and the Optimization Module.
- Supply Chain Analyst — primary user of Network Analysis and the benchmarking/evaluation dashboard.

- Procurement Officer — primary user of the Intelligent Decision Module.
- Driver — consumes the output of Route Optimization / Optimization Module on a simplified mobile view.
- Customer — consumes read-only tracking information derived from Route Optimization and Realtime status updates.

6. System Modules Overview

The table below summarizes the computational problem addressed by each of the five required modules, the candidate algorithms investigated, the selected solution, and the core data structures used, consistent with the coursework's Chapter 3–5 requirements.

Task	Module	Candidate Algorithms Investigated	Selected Algorithm	Core Data Structures
Task 1	Intelligent Route Optimization	Dijkstra's Algorithm, A* Search, Bellman-Ford	Dijkstra's Algorithm	Adjacency List (Linked List) + Min-Heap Priority Queue
Task 2	Intelligent Resource Allocation	0/1 Knapsack (DP), Greedy, Branch & Bound	0/1 Knapsack — Dynamic Programming	2-Dimensional DP Array + Arrays (weights/values)
Task 3	Network Analysis	Prim's Algorithm, Kruskal's Algorithm, Brandes' Algorithm	Prim's Algorithm (MST)	Min-Heap (edge selection) + Doubly Linked List (MST edges)
Task 4	Intelligent Decision Module	Decision Tree, k-Nearest Neighbours, Naive Bayes	Rule-based Decision Tree	Binary Tree + Queue/Stack (BFS/DFS traversal)
Task 5	Optimization Module (Intractable Problem)	Simulated Annealing, Genetic Algorithm, Exact DP	Simulated Annealing	Circularly Linked List (tour) + 2-Dimensional Distance Matrix

7. Detailed Module Design

7.1 Intelligent Route Optimization

Business Scenario: Finding the shortest, lowest-cost delivery path between two distribution hubs on the road network.

Selected Algorithm & Rationale: Dijkstra's Algorithm is selected as the primary solution because it guarantees the optimal shortest path on a graph with non-negative edge weights and is well suited to a relatively stable hub network. A* Search and Bellman-Ford are investigated and implemented as comparison baselines — A* to show the benefit of a heuristic on large graphs, and Bellman-Ford to illustrate correct behaviour on graphs that could contain negative adjustments (e.g., promotional routing discounts).

Data Structures: The road/hub network is stored as an Adjacency List (implemented with linked structures) because the graph is sparse (each hub connects to only a few neighbours), and a Min-Heap Priority Queue is used to always extract the next-nearest unvisited hub in $O(\log V)$, giving an overall complexity of $O((V + E) \log V)$.

Computational Complexity: Time Complexity: $O((V + E) \log V)$ using a binary heap; Space Complexity: $O(V + E)$ for the adjacency list plus $O(V)$ for the distance/heap arrays.

Supabase Tables Used: warehouses, network_edges, routes, route_stops, benchmark_logs.

System Features:

- Interactive map (Leaflet/Mapbox) showing warehouses/hubs as graph nodes and roads as weighted edges.
- "Find Best Route" action that runs Dijkstra's Algorithm between a selected origin and destination.
- Side-by-side comparison view against A* Search and Bellman-Ford for the same query (for the report's comparative analysis).
- Displays computed shortest distance, estimated time, and step-by-step path.

7.2 Intelligent Resource Allocation

Business Scenario: Maximizing the total value of cargo loaded onto a delivery vehicle without exceeding its maximum weight capacity.

Selected Algorithm & Rationale: The 0/1 Knapsack problem is solved using Dynamic Programming because it guarantees an exact, optimal solution for a bounded, discrete set of orders — a requirement for a decision-support tool making real allocation recommendations. A Greedy (value-to-weight ratio) heuristic and a Branch & Bound search are investigated as faster but non-guaranteed alternatives for comparison.

Data Structures: A 2-Dimensional DP Array (orders $\times$ remaining capacity) is used to build the table bottom-up, with standard Arrays holding the incoming item weights and values.

Computational Complexity: Time Complexity: $O(N \cdot W)$ where N is the number of orders and W is the vehicle's weight capacity; Space Complexity: $O(N \cdot W)$, reducible to $O(W)$ with a rolling array optimization.

Supabase Tables Used: orders, vehicles, allocations, benchmark_logs.

System Features:

- Order intake form capturing cargo weight and value per item.
- "Optimize Load" action that runs the 0/1 Knapsack DP to select the combination of orders maximizing value within a vehicle's weight limit.
- DP table visualizer (for demonstration/report evidence) showing how the optimal solution is built.
- Comparison mode against a Greedy (value/weight ratio) baseline to show optimality gap.

7.3 Network Analysis

Business Scenario: Determining the minimum-cost set of physical infrastructure links required to connect all regional warehouses/hubs.

Selected Algorithm & Rationale: Prim's Algorithm is selected to compute the Minimum Spanning Tree (MST) of the hub network, as it grows the tree incrementally from a starting hub and performs efficiently on the dense, geographically clustered hub graphs expected in this scenario. Kruskal's Algorithm is investigated as an edge-centric alternative, and Brandes' Algorithm (betweenness centrality) is discussed as a contrasting, more expensive technique for identifying critical hubs rather than minimal connections.

Data Structures: A Min-Heap is used to always select the lowest-weight edge connecting the growing tree to a new hub, and a Doubly Linked List stores the finalized MST edges to allow fast bidirectional traversal of the resulting supply-line network.

Computational Complexity: Time Complexity: $O(E \log V)$ using a binary heap; Space Complexity: $O(V + E)$.

Supabase Tables Used: warehouses, network_edges, mst_results, benchmark_logs.

System Features:

- Network graph view of all warehouses/hubs with editable edge distances/costs.
- "Compute Minimum Infrastructure" action that runs Prim's Algorithm to generate the Minimum Spanning Tree.
- Highlights selected MST edges on the map and reports total infrastructure cost.
- Comparison against Kruskal's Algorithm on the same dataset.

7.4 Intelligent Decision Module

Business Scenario: Classifying and recommending backup suppliers based on criteria such as delivery speed, cost, and historical defect rate.

Selected Algorithm & Rationale: A rule-based Decision Tree is selected because it produces transparent, explainable classification rules that a Procurement Officer can audit — an important property for a business-critical recommendation. k-Nearest Neighbours and Naive Bayes are investigated as comparison classifiers, evaluated on accuracy and interpretability trade-offs.

Data Structures: The trained model is stored as a Binary Tree, where internal nodes represent supplier criteria/thresholds and leaves represent the final risk classification; a Queue or Stack is used to evaluate/traverse the rules breadth-first or depth-first respectively.

Computational Complexity: Training: $O(n \cdot d \cdot \log n)$ typical for a depth-bounded tree (n records, d features); Classification: $O(\log n)$ average case per supplier for a balanced tree, $O(n)$ worst case for a degenerate tree.

Supabase Tables Used: suppliers, decision_rules, benchmark_logs.

System Features:

- Supplier registry with editable criteria (cost, delivery speed, defect rate).
- "Classify Supplier Risk" action that traverses the trained Decision Tree and returns Low/Medium/High risk with the reasoning path.
- Admin tool to retrain / rebuild the Decision Tree from updated supplier history.
- Comparison view against a k-NN classifier for the same supplier dataset.

7.5 Optimization Module (Intractable Problem)

Business Scenario: Solving the intractable multi-stop Vehicle Routing / Travelling Salesperson Problem for a vehicle's full daily set of delivery stops.

Selected Algorithm & Rationale: Simulated Annealing is selected as the primary heuristic because it can escape local optima by probabilistically accepting worse intermediate tours early in the search, gradually cooling toward a highly optimized route — directly addressing Learning Outcome 3 on approximation methods for intractable problems. A Genetic Algorithm is investigated as an alternative population-based metaheuristic, and an Exact Dynamic Programming (Held-Karp) solution is used as a ground-truth baseline for small instances.

Data Structures: The delivery sequence is stored in a Circularly Linked List, which naturally models a closed-loop route that returns to the starting depot and allows $O(1)$ adjacent-node swaps during the annealing process; all pairwise distances are pre-computed into a 2-Dimensional Distance Matrix.

Computational Complexity: Time Complexity: $O(k \cdot n)$ per run for k annealing iterations over n stops (in practice k is chosen based on a cooling schedule); Space Complexity: $O(n^2)$ for the distance matrix plus $O(n)$ for the tour.

Supabase Tables Used: vehicles, orders, routes, route_stops, benchmark_logs.

System Features:

- "Plan Daily Delivery Run" action that takes all of a vehicle's assigned stops and runs Simulated Annealing to find a near-optimal closed-loop route.
- Live progress indicator showing the annealing temperature schedule and improving tour cost.
- Final route rendered on the map as a closed loop returning to the depot.
- Comparison against a Genetic Algorithm run and (for small n) an exact DP solution, with cost/runtime shown side by side.

8. Database Design (Supabase / PostgreSQL)

All persistent data is stored in Supabase-managed PostgreSQL. Supabase Auth's built-in auth.users table is extended by a profiles table carrying the application role. Row Level Security (RLS) policies restrict each role's access — for example, a driver can only SELECT rows in route_stops where the linked route belongs to their own driver_id.

Table	Key Columns	Purpose
profiles	id, full_name, email, role, phone	Extends Supabase auth.users; stores role (admin, dispatcher, analyst, driver, procurement, customer).
warehouses	id, name, latitude, longitude, capacity	Master data for distribution centres / hubs used by Route Optimization and Network Analysis.
network_edges	id, hub_from, hub_to, distance_km, build_cost	Weighted edges between hubs; source graph for Dijkstra's Algorithm and Prim's MST.
vehicles	id, plate_no, max_weight_kg, status, driver_id	Fleet data used by the Resource Allocation (Knapsack) module.
drivers	id, profile_id, license_no, availability	Driver roster linked to vehicles and routes.
orders	id, customer_id, origin_id, destination_id, weight_kg, value, priority, status	Customer orders; input to Route Optimization, Resource Allocation and the Decision module.
routes	id, order_id / batch_id, algorithm_used, total_distance, total_cost, created_at	Stores the output route/sequence generated by Dijkstra or Simulated Annealing.
route_stops	id, route_id, stop_order, warehouse_id, status, eta	Ordered stop list of a computed route (implemented as a circular sequence for multi-stop runs).
allocations	id, vehicle_id, order_id, allocated_weight, allocated_value	Result of the 0/1 Knapsack allocation for a given vehicle/day.
mst_results	id, run_id, edge_id, included	Stores the edges selected by Prim's Algorithm for a given Network Analysis run.

Table	Key Columns	Purpose
suppliers	id, name, cost_score, delivery_speed_score, defect_rate, risk_class	Supplier attributes evaluated by the Decision Tree classifier.
decision_rules	id, node_id, parent_id, criterion, threshold, is_leaf, classification	Serialized nodes of the trained Decision Tree (binary tree structure).
benchmark_logs	id, module_name, algorithm_name, input_size, execution_time_ms, memory_kb, run_at	Captures experimental performance data (varying dataset sizes) for Chapters 7–9 evaluation charts.

8.1 Entity Relationship Notes

- orders references warehouses (origin/destination) and profiles (customer).
- routes and route_stops together represent the output of both the Route Optimization module (point-to-point) and the Optimization Module (multi-stop, circular sequence).
- allocations links vehicles to orders as produced by the Resource Allocation module.
- mst_results links to network_edges to record which edges Prim's Algorithm selected for a given analysis run.
- decision_rules is a self-referencing table (parent_id) representing the Binary Tree structure of the trained Decision Tree.
- benchmark_logs is written to by every module after each run and is the primary data source for the coursework's experimental evaluation charts (Chapters 7–9).

9. System Features Summary

The following consolidated feature list represents the full functional scope of SmartLogix as delivered across all modules and dashboards.

9.1 Core Platform Features

- Secure role-based login and session management (Supabase Auth).
- Role-specific dashboards for Admin, Dispatcher, Analyst, Driver, Procurement Officer and Customer.
- Master data management for warehouses, vehicles, drivers and the hub network.
- Real-time order and delivery status updates (Supabase Realtime) across dispatcher, driver and customer views.
- Exportable performance/benchmark reports (PDF/CSV via Supabase Storage) for use in the coursework reports.

9.2 Module-Specific Features

Task 1 — Intelligent Route Optimization

- Interactive map (Leaflet/Mapbox) showing warehouses/hubs as graph nodes and roads as weighted edges.
- "Find Best Route" action that runs Dijkstra's Algorithm between a selected origin and destination.

- Side-by-side comparison view against A* Search and Bellman-Ford for the same query (for the report's comparative analysis).
- Displays computed shortest distance, estimated time, and step-by-step path.

Task 2 — Intelligent Resource Allocation

- Order intake form capturing cargo weight and value per item.
- "Optimize Load" action that runs the 0/1 Knapsack DP to select the combination of orders maximizing value within a vehicle's weight limit.
- DP table visualizer (for demonstration/report evidence) showing how the optimal solution is built.
- Comparison mode against a Greedy (value/weight ratio) baseline to show optimality gap.

Task 3 — Network Analysis

- Network graph view of all warehouses/hubs with editable edge distances/costs.
- "Compute Minimum Infrastructure" action that runs Prim's Algorithm to generate the Minimum Spanning Tree.
- Highlights selected MST edges on the map and reports total infrastructure cost.
- Comparison against Kruskal's Algorithm on the same dataset.

Task 4 — Intelligent Decision Module

- Supplier registry with editable criteria (cost, delivery speed, defect rate).
- "Classify Supplier Risk" action that traverses the trained Decision Tree and returns Low/Medium/High risk with the reasoning path.
- Admin tool to retrain / rebuild the Decision Tree from updated supplier history.
- Comparison view against a k-NN classifier for the same supplier dataset.

Task 5 — Optimization Module (Multi-stop VRP/TSP)

- "Plan Daily Delivery Run" action that takes all of a vehicle's assigned stops and runs Simulated Annealing to find a near-optimal closed-loop route.
- Live progress indicator showing the annealing temperature schedule and improving tour cost.
- Final route rendered on the map as a closed loop returning to the depot.
- Comparison against a Genetic Algorithm run and (for small n) an exact DP solution, with cost/runtime shown side by side.

10. End-to-End System Workflow

The following sequence illustrates how a typical delivery cycle exercises all five modules together, demonstrating the required system integration.

1. A Dispatcher logs into the Next.js web application (Supabase Auth) and creates a new customer order, specifying origin, destination, cargo weight and value.
2. The system calls the Route Optimization service (Dijkstra's Algorithm) to compute the shortest point-to-point path between origin and destination hubs, storing the result in routes and route_stops.
3. When multiple orders are pending for a vehicle, the Dispatcher runs Resource Allocation (0/1 Knapsack DP) to select the optimal combination of orders that fit the vehicle's weight capacity while maximizing delivered value.

4. For the day's full delivery run (multiple stops for one vehicle), the system invokes the Optimization Module (Simulated Annealing) to sequence all assigned stops into a near-optimal closed-loop tour, stored using a circularly linked route_stops sequence.
5. In parallel, a Supply Chain Analyst runs Network Analysis (Prim's Algorithm) periodically to verify that the warehouse/hub network's infrastructure (edge set) remains cost-minimal as new hubs are added.
6. When stock levels trigger a need for a backup supplier, a Procurement Officer consults the Intelligent Decision Module, which classifies available suppliers via the trained Decision Tree and recommends the lowest-risk option.
7. Drivers view their assigned optimized route on a mobile-friendly dashboard and update delivery status per stop in real time via Supabase Realtime, which is reflected instantly on the Dispatcher's live dashboard and the Customer's tracking page.
8. Every algorithm execution is timed and logged to benchmark_logs; the Analyst exports comparison charts (execution time, memory, scalability) for use in the coursework's experimental evaluation chapters.

11. Non-Functional Requirements

Requirement	Description
Performance	Core algorithm responses (Dijkstra, Knapsack) should return within 2 seconds for graphs/datasets up to 500 nodes/items in the demo environment.
Scalability	Supabase Postgres and stateless Next.js API routes allow horizontal scaling; heavy Simulated Annealing / Genetic Algorithm runs can be queued to a background Edge Function.
Security	Supabase Auth issues JWTs; Row Level Security policies restrict each role to its permitted tables/rows (e.g., drivers can only see their own routes).
Availability	Vercel + Supabase managed hosting provide high availability with automatic daily database backups.
Usability	Role-based dashboards (Admin, Dispatcher, Analyst, Driver, Procurement, Customer) present only the features relevant to that user.
Maintainability	Each of the five DSA modules is implemented as an isolated service module (e.g., /lib/algorithms/dijkstra.ts) with unit tests, so algorithms can be swapped or benchmarked independently.
Auditability	All algorithm runs are logged to benchmark_logs with timestamp, input size, execution time and memory, supporting the experimental evaluation chapters.

12. Experimental Performance Evaluation Plan

To satisfy the coursework's requirement for experimental evidence (Chapters 7–9), each module will be benchmarked across increasing dataset sizes, with results logged to the benchmark_logs table and visualized as line/bar charts comparing candidate algorithms.

Module	Algorithms Compared	Dataset Sizes	Metrics Captured
Route Optimization	Dijkstra vs A* vs Bellman-Ford	50, 200, 500, 1000 nodes	Execution time, nodes expanded, memory

Module	Algorithms Compared	Dataset Sizes	Metrics Captured
Resource Allocation	0/1 Knapsack DP vs Greedy vs Branch & Bound	20, 100, 500, 1000 items	Execution time, solution value (optimality gap), memory
Network Analysis	Prim's vs Kruskal's	20, 100, 300 hubs / edges	Execution time, total MST cost, memory
Decision Module	Decision Tree vs k-NN vs Naive Bayes	50, 200, 1000 supplier records	Classification accuracy, execution time
Optimization Module	Simulated Annealing vs Genetic Algorithm vs Exact DP	10 (exact-feasible), 50, 200, 1000 stops	Tour cost, execution time, convergence behaviour

Each benchmark run should be repeated multiple times (e.g., 5–10 runs per data point) and averaged to reduce measurement noise, with results plotted as execution time vs. input size and memory vs. input size for each competing algorithm, directly supporting the comparative analysis and critical evaluation chapters of both the Individual and Group Reports.

13. Deployment Architecture

- Source Control: Git repository (e.g., GitHub) shared by all group members, with one branch per module/task.
- Frontend/Application Hosting: Vercel, connected to the GitHub repository for automatic build and deployment on every merge to main.
- Backend/Database: Supabase Cloud project providing PostgreSQL, Auth, Storage and Realtime — no separate server management required.
- Environment Configuration: Supabase URL and API keys stored as environment variables in Vercel; separate development and production Supabase projects recommended to protect coursework demo data.
- CI/CD: GitHub Actions (optional) to run unit tests on the algorithm service modules before deployment.

14. Conclusion

SmartLogix demonstrates how five classically distinct computational problems — shortest-path routing, constrained resource allocation, minimum spanning tree network design, rule-based classification, and heuristic optimization for an intractable routing problem — can be unified into a single, coherent Intelligent Decision Support System for logistics. By implementing the system on Next.js and Supabase, the group benefits from a modern, easily deployable full-stack architecture while still fully satisfying the coursework's requirements: investigating and comparing multiple candidate algorithms per module, justifying the final algorithm and data-structure selection, analyzing computational complexity, and gathering experimental evidence of performance across varying input sizes.